



Universidade de Brasília
Departamento de Ciência da Computação
Teleinformática e Redes 1 — CIC0235

Simulador TR1 — Camadas Física e de Enlace

Anna Luiza Novaes Jansen Silva — 232024572
João Pedro Borges Saraiva — 231025280
Luís Eduardo Curi Serra — 251033574

Professor: Marcelo Antonio Marotta

25 de junho de 2026

1 Introdução

O objetivo deste trabalho é **simular as camadas Física e de Enlace** de uma rede de computadores, reproduzindo todo o caminho de uma mensagem de texto desde a digitação no **transmissor (TX)** até a sua recuperação no **receptor (RX)**. O problema central é representar a informação como *sinaletrico* (valores de tensão/potência, V/W), transmiti-lo por um meio que insere **ruído gaussiano** e, ainda assim, recuperar a mensagem original por meio dos protocolos de enquadramento, detecção e correção de erros.

O simulador foi desenvolvido em **Python 3**, sem bibliotecas prontas para os algoritmos centrais (CRC, Hamming, modulações): NumPy e Matplotlib são usados apenas para manipular vetores e plotar sinais [5, 1]. O fluxo completo é:

$$\text{texto} \rightarrow \text{bits} \rightarrow \text{Enlace (EDC/ECC + enquadramento)} \rightarrow \text{Física (sinal V/W)} \rightarrow \boxed{\text{meio com ruído } n(x, \sigma)} \rightarrow \text{Física RX} \rightarrow \text{Enlace RX} \rightarrow \text{bits} \rightarrow \text{texto}.$$

Em conformidade com a Figura 1 do enunciado [3], o TX e o RX são **processos separados**, executados como duas janelas gráficas independentes que se comunicam por um **socket TCP local**, o qual representa o meio de comunicação onde o ruído é aplicado.

2 Implementação

2.1 Organização e arquitetura

O código foi modularizado em arquivos com responsabilidades bem definidas (Tabela 1). A comunicação entre camadas usa uma convenção única de dados: **bits** são listas de inteiros 0/1 (`list[int]`); o **sinal** é um `np.ndarray` de valores reais em V/W, com AMOSTRAS_POR_BIT = 100 amostras por bit/símbolo. Para a interface escolher as técnicas em tempo de execução, cada camada expõe **despachantes** que mapeiam a string da GUI para a implementação correspondente (`coder/decoder` e `modulator/demodulator` na física; `adicionar_controle_erro` e `aplicar_enquadramento` no enlace), evitando código duplicado.

Arquivo	Responsabilidade
CamadaFisica.py	Modulação digital, por portadora, amostragem e ruído.
CamadaEnlace.py	Enquadramento, detecção (EDC) e correção (ECC) de erros.
Transmissor.py	Fluxo TX: texto $\rightarrow$ bits $\rightarrow$ enlace $\rightarrow$ física.
Receptor.py	Fluxo RX: sinal $\rightarrow$ física $\rightarrow$ enlace $\rightarrow$ texto.
InterfaceGrafica.py	GUI GTK 3 + Matplotlib (TX e RX).
Simulador.py	Rotina que orquestra TX $\rightarrow$ meio $\rightarrow$ RX.
utils.py	Conversões base (<code>int</code> $\leftrightarrow$ bits, texto $\leftrightarrow$ bits, padding).

Tabela 1: Organização dos módulos do simulador.

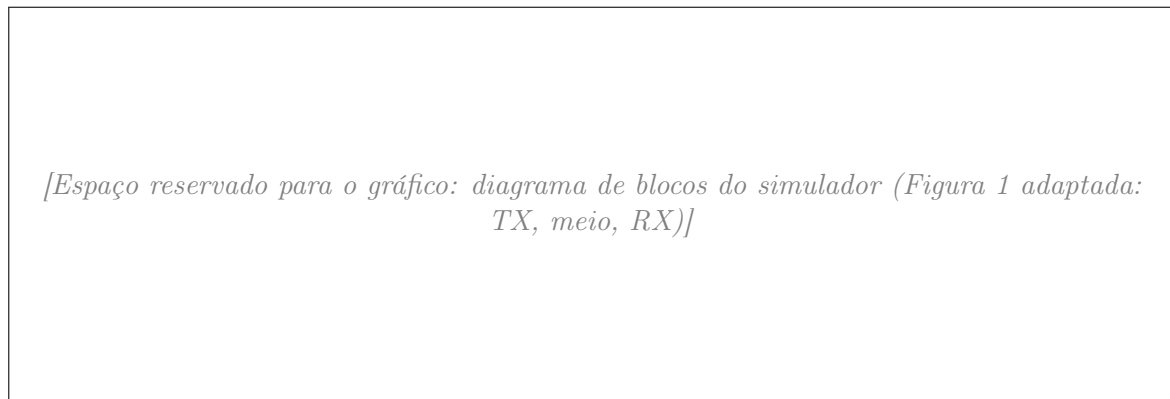


Figura 1: Diagrama de desenvolvimento adotado, com TX e RX em processos separados ligados pelo meio (socket TCP) onde o ruído é inserido.

2.2 Camada Física

A camada física converte os bits em sinal V/W (níveis $\pm 1,5$ V). Foram implementadas as três **modulações digitais** (banda-base) e as cinco **modulações por portadora** exigidas:

- **NRZ-Polar**: bit 1 $\rightarrow +V$; bit 0 $\rightarrow -V$.
- **Manchester** (IEEE 802.3): transição no meio do bit (1 = alto $\rightarrow$ baixo; 0 = baixo $\rightarrow$ alto).
- **Bipolar (AMI)**: bit 0 $\rightarrow 0$ V; bit 1 alterna $+V / -V$.
- **ASK / FSK / PSK**: chaveamento de amplitude, frequência e fase de uma portadora senoidal.
- **QPSK**: 2 bits por símbolo (4 fases); **16-QAM**: 4 bits por símbolo (constelação I/Q 4×4).

A demodulação é tolerante ao ruído: o banda-base decide pela *média* das amostras do bit; ASK, FSK, PSK e QPSK usam *correlação* com as portadoras de referência; o 16-QAM estima as componentes *I* e *Q* e escolhe o ponto da constelação de *menor distância euclidiana*.

Cenário de teste: Os testes são conduzidos com um efeito por vez, sempre será enviada a palavra *TR1!* para preservar o tamanho máximo de 40bits(3200 pontos no gráfico, pois para cada bit são usadas 100 amostras) fixado no gráfico da interface gráfica, assim facilitando a visualização da informação. Adicionalmente vale notar que na implementação a modulação por portadora tem predominância sobre a modulação digital.

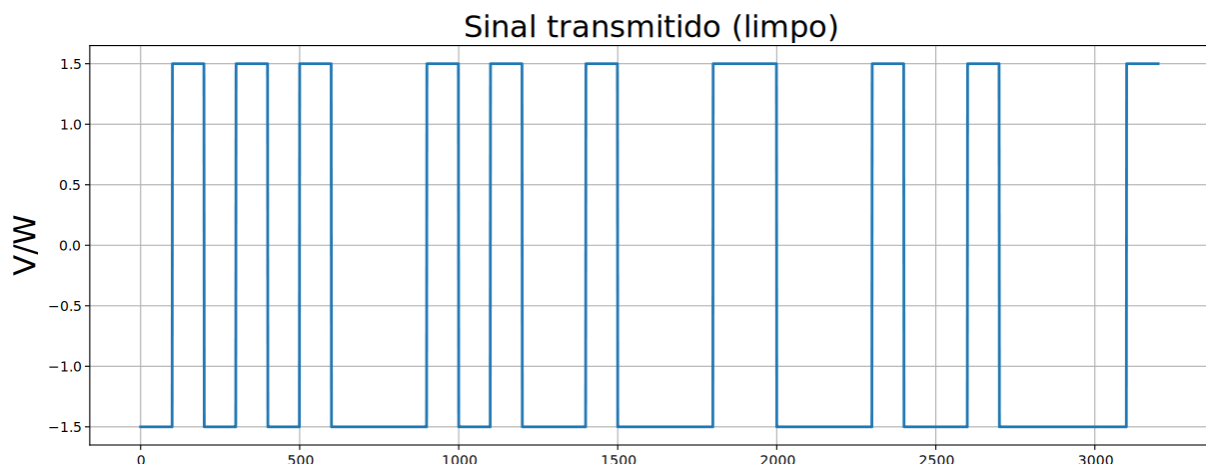


Figura 2: Sinal da modulação digital NRZ-Polar (banda-base).

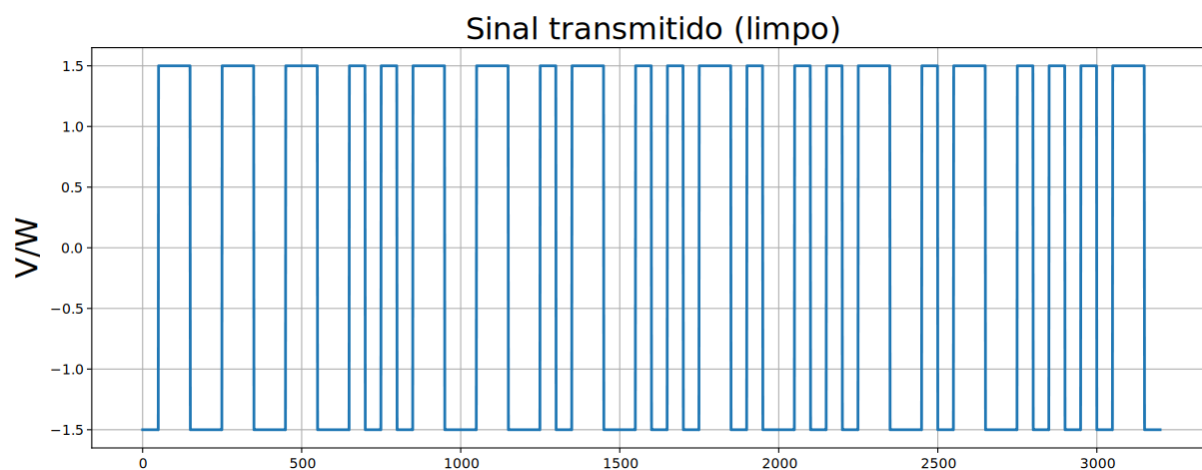


Figura 3: Sinal da modulação digital Manchester (banda-base).

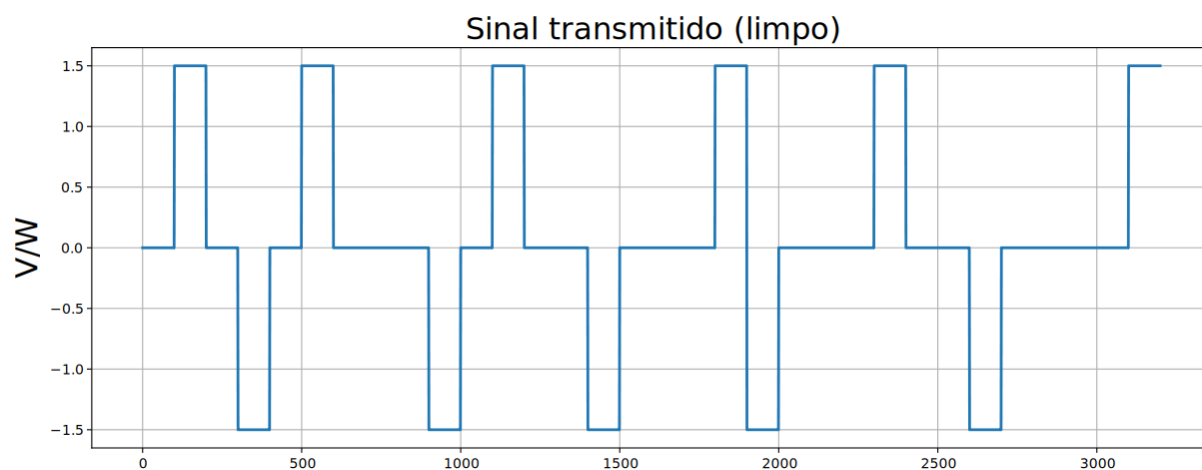


Figura 4: Sinal da modulação digital Bipolar (banda-base).

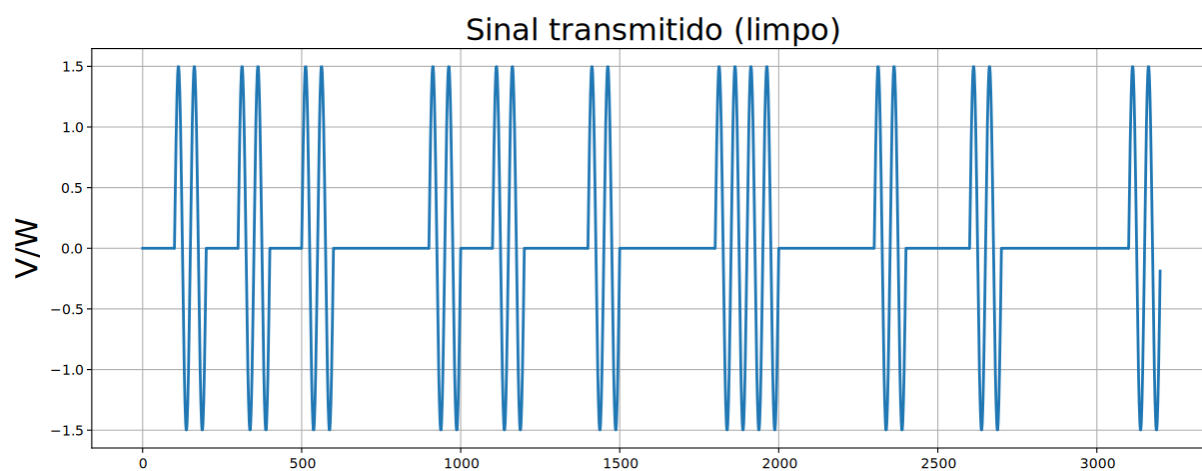


Figura 5: Sinal da modulação por portadora ASK.

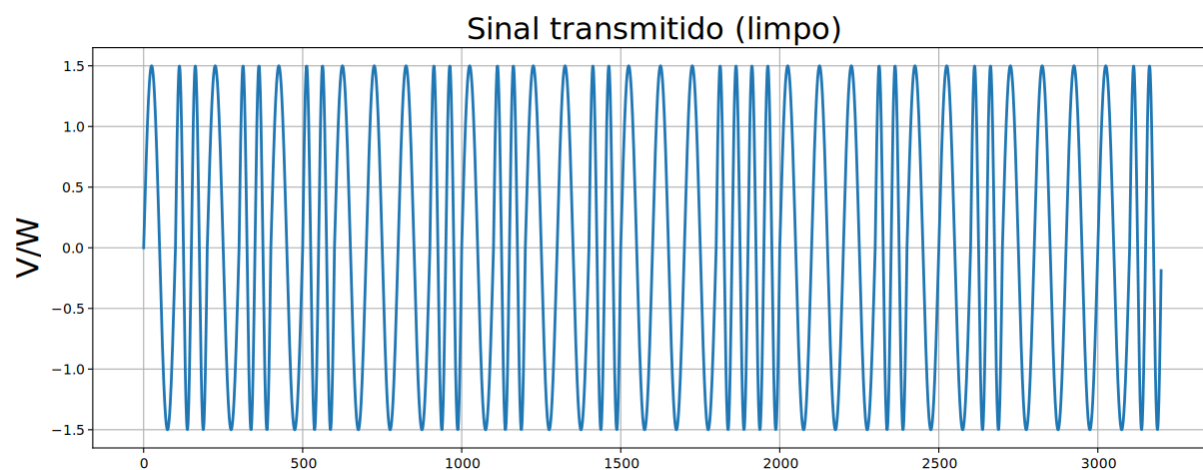


Figura 6: Sinal da modulação por portadora FSK.

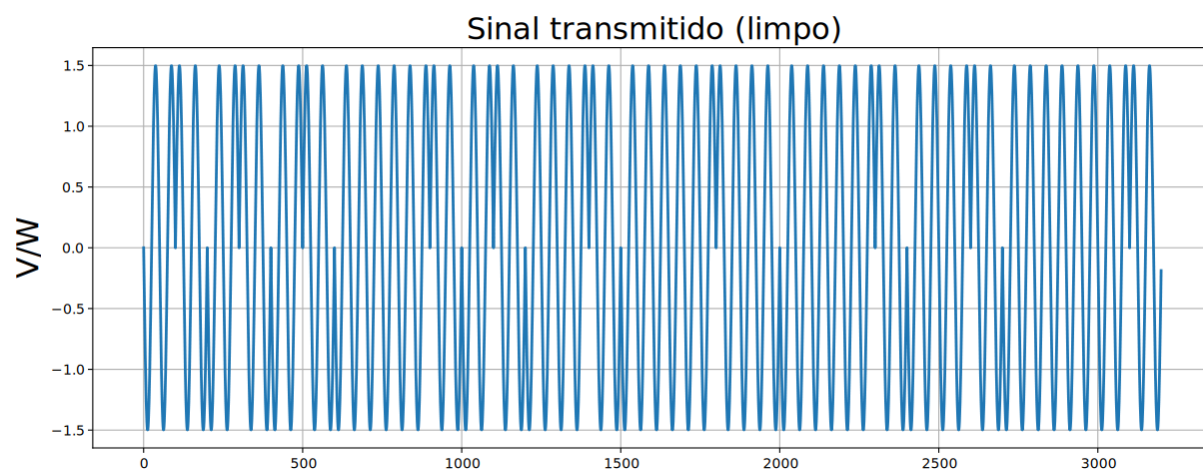


Figura 7: Sinal da modulação por portadora PSK.

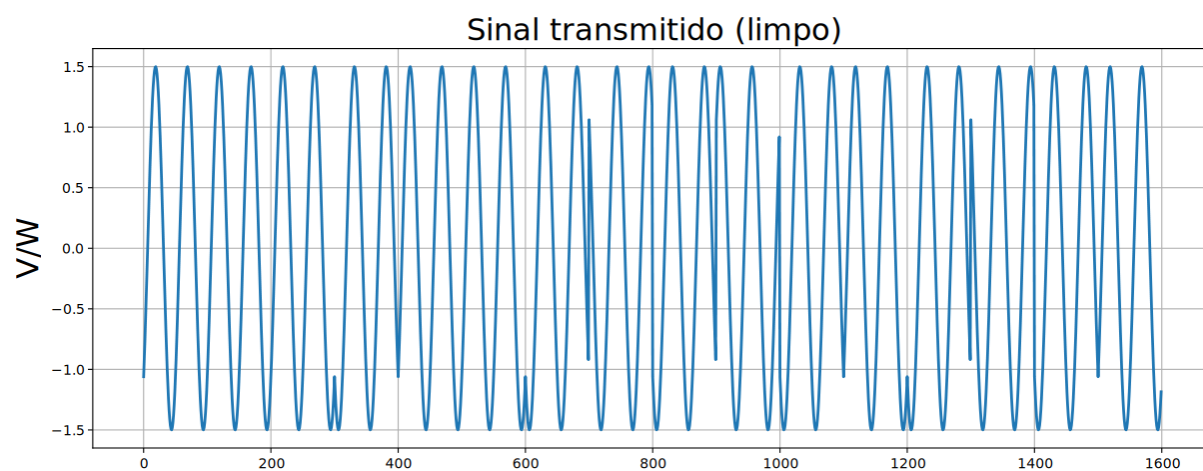


Figura 8: Sinal da modulação por portadora QPSK.

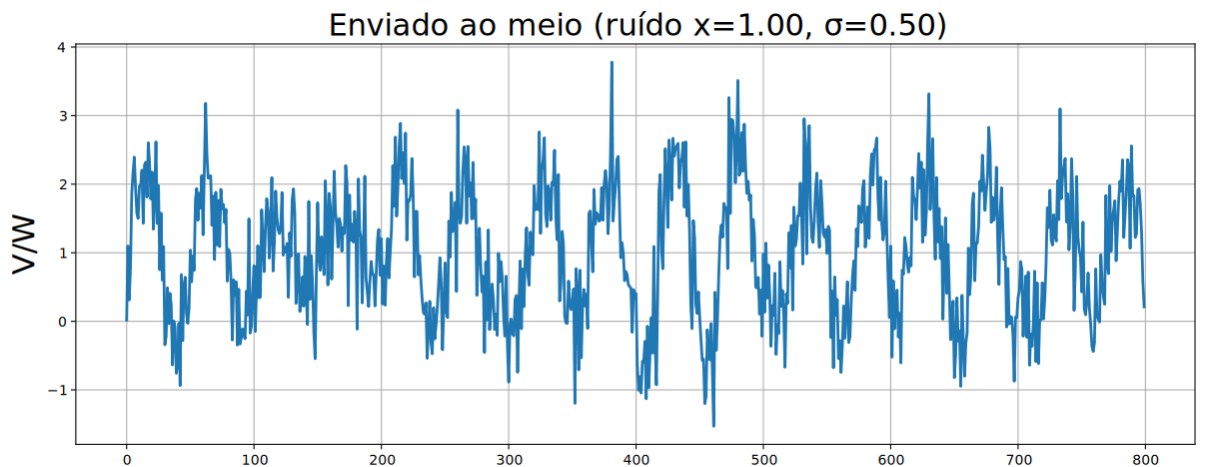
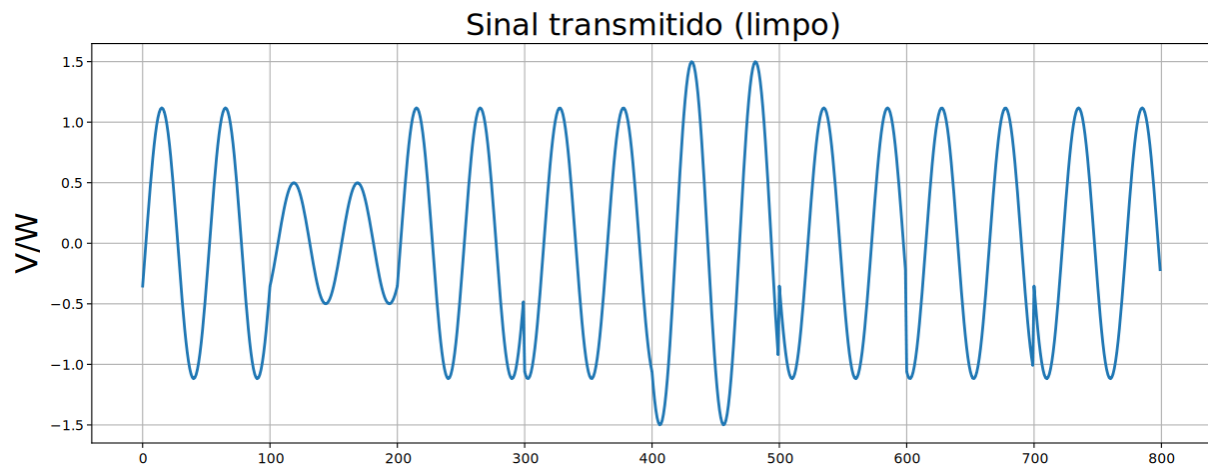

 Figura 10: Portadora com Ruído: 16-QAM, $\mu = 1$, $\sigma = 0.5$


Figura 9: Sinal da modulação por portadora 16-QAM.

Para o QPSK cada portadora carrega 2bits, logo o gráfico contém a metade do número de amostras e para o 16-QAM, carrega 4bits, assim são usados um quarto do número de amostras. Na sequência segue o exemplo com ruído

2.3 Camada de Enlace

A camada de enlace opera sobre bits. No TX o pipeline é *payload* → *adiciona EDC/ECC* → *enquadra*; no RX, o inverso. Foram implementados:

- **Enquadramento:** (i) *contagem de caracteres*, com cabeçalho de **16 bits** indicando o tamanho do quadro; (ii) *inserção de bytes* (byte stuffing) com flag **0x7E** e escape **0x7D**; (iii) *inserção de bits* (bit stuffing), inserindo um 0 após cinco 1 consecutivos.
- **Deteção (EDC):** bit de *paridade par*; *checksum* por soma em complemento de um com *carry* circular; e **CRC-32** implementado manualmente por divisão polinomial (gerador IEEE 802, 0x04C11DB7), *sem* usar *zlib*.
- **Correção (ECC):** código de **Hamming**, com bits de paridade nas posições potência de 2; no RX, a síndrome localiza e corrige 1 bit invertido por quadro.

Cenário de teste: Foi alterado o número de amostras para 1 de forma a ficar visível o

número de bits do quadro. Valor de bits por quadro foi fixado em 8. Além disso, será transmitido apenas o caractere "T" por simplicidade "01010100".

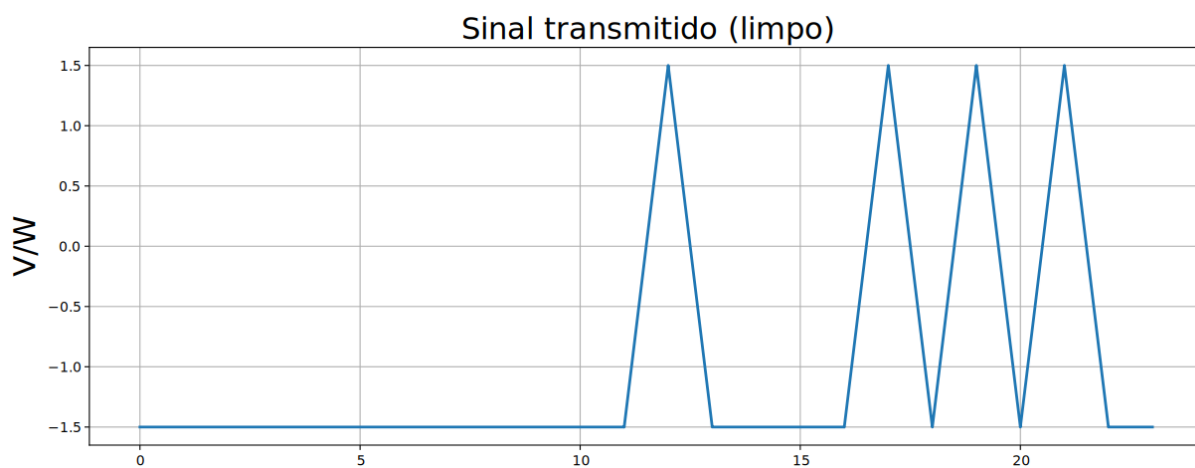


Figura 11: Enquadramento por contagem de caracteres - cabeçalho de 16 bits + payload

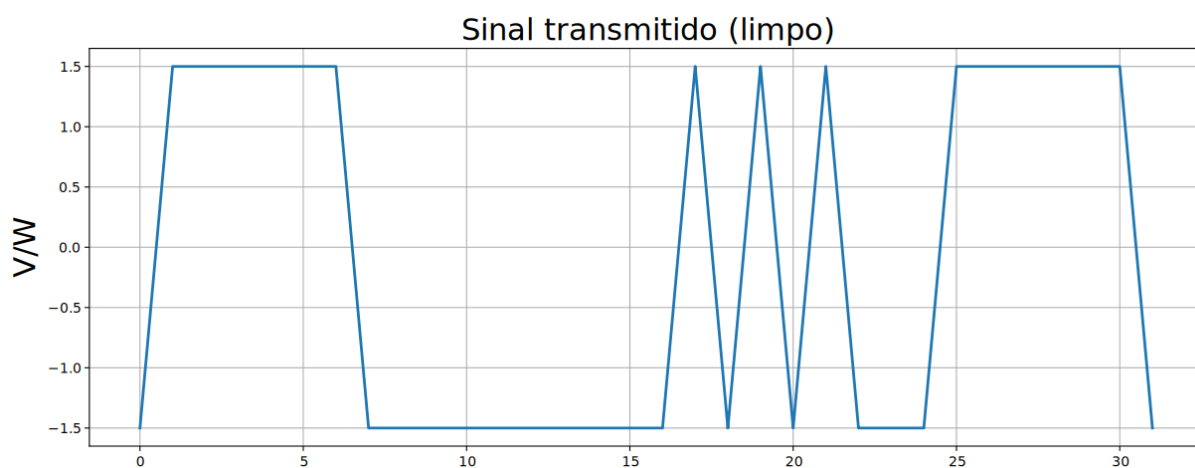


Figura 12: Enquadramento por inserção de bytes

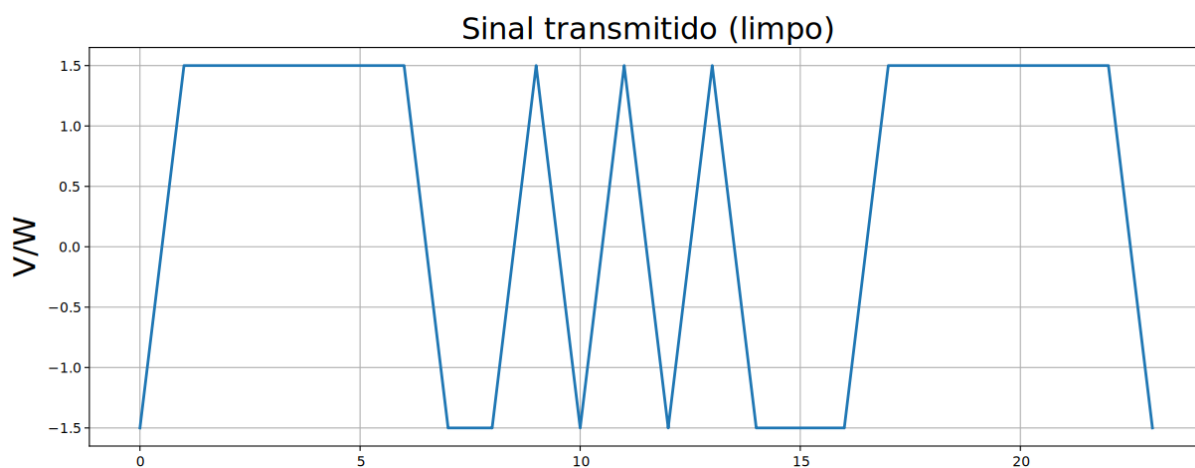


Figura 13: Enquadramento por inserção de bits

Texto & Bits	Gráficos
Texto enviado (TX): T Texto recuperado (RX): U Bits recebidos (RX): 01010101 Status EDC: X erro detectado	Mensagem: T Tam. máx. quadro: 8 - + Enquadramento: Contagem de caracteres ▾ Detecção/Correção: Paridade ▾ Tam. checksum (k): 8 - + Modulação digital: Bipolar ▾ Modulação portadora: 16-QAM ▾ Ruído — média (μ): 2.00 - + Ruído — desvio padrão (σ): 2.00 - + ▶ TRANSMITIR Enviado ao Receptor ✓

Figura 14: Demonstração de detecção de erro por paridade

2.4 Meio de comunicação e ruído

A função `aplicar_ruído` soma a cada amostra do sinal um ruído gaussiano $n(x, \sigma)$ gerado com `random.gauss` (biblioteca padrão), com média x e desvio σ configuráveis na GUI. O ruído permite demonstrar, na prática, a atuação dos protocolos de detecção e correção.

2.5 Decisões de projeto e divergências do diagrama inicial

Durante a implementação, alguns pontos omissos no enunciado exigiram decisões de projeto, e o produto final divergiu em alguns aspectos do diagrama inicialmente esboçado:

1. **Tamanho máximo do quadro fixado em 212 bits** (`TAMANHO_QUADRO`, múltiplo de 8). O *payload* é mantido separado do cabeçalho e do EDC/ECC dentro do quadro, decisão que organizou o fatiamento da mensagem e o enquadramento.
2. **O cabeçalho de contagem conta bits, não caracteres** (16 bits), para casar com o fato de todo o projeto trafegar a informação em bits.
3. **EDC ou ECC, nunca os dois simultaneamente** por quadro. A interface permite escolher *uma* técnica de detecção *ou* a correção (Hamming), tornando-os mutuamente exclusivos — convenção definida pelo grupo para o caso omissos no enunciado.
4. **O ruído é aplicado fora do TX e do RX**. Diferentemente de uma leitura literal do diagrama, o ruído não é inserido dentro de `Transmissor.py` nem `Receptor.py`: ele é aplicado no *canal* (lado do TX, imediatamente antes do envio pelo socket), mantendo o transmissor e o receptor “puros”.
5. **Meio implementado como socket TCP** (127.0.0.1:5001) e **GUI em duas instâncias** (`-modo tx` e `-modo rx`), em vez de uma janela única. Essa escolha materializa de forma clara o requisito de TX e RX como processos separados.

2.6 Uso de Inteligência Artificial

Por transparência, registra-se que o **guia inicial de construção** do projeto foi elaborado com auxílio de IA, com o intuito de *deixar mais claro para o grupo* como dividir as tarefas e qual

o passo a passo de desenvolvimento. A IA também foi usada para **montar casos de teste** das funções individuais das camadas de enlace e física e para **apontar possíveis erros de lógica**. Entretanto, **toda a lógica de implementação foi desenvolvida pelo grupo**: a IA não foi utilizada para corrigir erros — apenas para identificar possíveis falhas de lógica, que foram *depuradas e corrigidas pelos próprios integrantes*.

3 Membros e atividades

Integrante	Atividades desenvolvidas
Luís Eduardo Curi Serra	Interface gráfica, camada física e correção de erros (Hamming, ECC).
Anna Luiza Novaes Jansen Silva	Transmissor, simulador e enquadramento (quadros).
João Pedro Borges Saraiva	Receptor e detecção de erros (paridade, checksum, CRC-32 — EDC).

Tabela 2: Divisão de atividades entre os membros do grupo.

4 Conclusão

O simulador atendeu aos requisitos do enunciado, transmitindo corretamente a mensagem através das camadas física e de enlace e demonstrando a detecção e a correção de erros sob ruído. A parte mais **difícil** foi o *front-end* da interface gráfica, principalmente pela falta de conhecimento prévio do grupo em GTK e pelo esforço para deixá-la minimamente estética. A **camada física** mostrou-se objetiva e direta de implementar. Já a **camada de enlace** foi a mais desafiadora em termos de decisões de projeto — em especial a escolha do tamanho dos quadros e se o *payload* deveria ou não ser separado dentro do quadro.

Como balanço geral, o trabalho **auxiliou bastante o grupo** a concretizar conceitos da camada de enlace que, até então, eram bastante abstratos, tornando tangíveis o enquadramento e os mecanismos de controle de erros vistos em sala [2, 4].

Referências

- [1] John D. Hunter and The Matplotlib Development Team. *Matplotlib: Visualization with Python — Documentation*, 2024.
- [2] Marcelo Antonio Marotta. Teleinformática e Redes 1 (CIC0235) — Material e slides da disciplina. Moodle / Aprender, Universidade de Brasília, 2025. Slides de Modulação, Enquadramento, Detecção e Correção de erros.
- [3] Marcelo Antonio Marotta. Trabalho Final de Teleinformática e Redes 1 — Enunciado. Universidade de Brasília, Departamento de Ciência da Computação, 2025.
- [4] Andrew S. Tanenbaum and David J. Wetherall. *Redes de Computadores*. Pearson Prentice Hall, 5 edition, 2011.
- [5] The NumPy Community. *NumPy Documentation*, 2024.